Engineering with Computers (2026) 42:101
https://doi.org/10.1007/s00366-026-02334-0

ORIGINAL ARTICLE

Learning-based toolpath planning for bed-based additive manufacturing

R. Schmeitz[1,2] · R. J. M. Wolfs[2,3] · J. J. C. Remmers[1,2]

Received: 14 January 2026 / Accepted: 16 April 2026

Abstract
Conventional toolpath strategies in additive manufacturing often ignore the underlying physics, leading to defects and inefficiency. In this paper, we present a reinforcement learning framework that couples policy learning directly with finite-element thermal simulations for powder bed fusion, enabling agents to learn adaptive toolpaths from spatially resolved temperature feedback. Using procedurally generated geometries to ensure robust generalization, we train Deep Q-Network and Proximal Policy Optimization agents in both simplified and thermally coupled environments. Our results demonstrate that learned policies outperform conventional zigzag baselines in terms of speed and generalize effectively to irregular geometries, discovering emergent thermal strategies such as outside-to-inside scanning patterns.

Keywords Additive manufacturing · Toolpath planning · Reinforcement learning · Finite element thermal simulation · Procedural generation

1 Introduction

Additive manufacturing (AM) processes such as powder bed fusion (PBF) rely on localized energy to selectively melt and solidify material in a layer-wise manner. Because this localized melting creates complex thermal gradients, achieving optimal part quality requires process optimization that incorporates the governing heat transfer and solidification physics to mitigate the creation of defects such as lack of fusion, overheating, warping, and residual stress [1, 2]. However, in practice, process parameters including laser power, scan speed, hatch spacing, and layer timing are typically chosen based on empirical design rules, lookup tables, or process maps derived from prior experimentation [3]. While such rule-based strategies have achieved reliable results across a wide range of materials and geometries, static parameter sets often fail to account for local variations in geometry or environmental conditions as the demand for higher build quality and efficiency increases [4, 5].

In addition to process parameters, the spatial sequencing of the laser path strongly influences local temperature evolution and mechanical integrity. Standard raster or zigzag strategies, while simple and widely used, do not take into account local thermal variations [6]. As a result, despite numerous mitigation efforts, most industrial systems still rely on trial-and-error tuning and operator experience to balance speed, accuracy, and reliability [7].

To overcome these limitations, physics-based optimization frameworks have been developed. Approaches such as open-loop optimal control, dynamic programming, and model predictive control incorporate thermal or mechanical feedback into process planning and toolpath sequencing. For example, Khrenov et al. [8] formulated an optimal control framework to minimize layer-level thermal variance in electron beam powder bed fusion, while Kim and Zohdi [9] used dynamic programming with deep learning acceleration to optimize scanning sequences in selective laser sintering. Jiang et al. [10] reviewed path planning strategies

✉ R. Schmeitz
r.schmeitz@tue.nl

R. J. M. Wolfs
r.j.m.wolfs@tue.nl

J. J. C. Remmers
j.j.c.remmers@tue.nl

1 Department of Mechanical Engineering, Eindhoven University of Technology, Eindhoven, The Netherlands

2 Eindhoven AI Systems Institute, Laplace 32, 5612 AJ Eindhoven, The Netherlands

3 Department of the Built Environment, Eindhoven University of Technology, Eindhoven, The Netherlands

Published online: 06 May 2026

highlighting trade-offs between thermal stability, geometric accuracy, and build time. Although these methods embed process physics and constraints effectively, they remain computationally demanding and require extensive calibration to stay robust under varying process conditions [11]. Their reliance on high-fidelity models and limited generalization across geometries, materials, and machines restrict scalability.

More recently, machine learning, particularly reinforcement learning (RL), has emerged as a data-driven alternative to model-based optimization, capable of learning control policies directly through interaction with simulated environments. Early studies demonstrated RL for thermal control in laser powder bed fusion, where the agent adjusted laser power and scanning velocity to stabilize the melt pool [12]. Subsequent work extended this concept to toolpath generation [13], process parameter optimization [14], and geometry-agnostic path design using numerical models [15]. Recent contributions have further integrated uncertainty-aware and defect-mitigation terms into RL-based control frameworks [16, 17], underscoring the growing potential of RL for adaptive process control.

Despite these advances, most studies rely on surrogate or reduced-order thermal models to remain computationally feasible. These environments encode simplified assumptions about additive manufacturing, such as linear relationships between scanning angle and melt pool depth or heuristic rewards for hatch uniformity [15, 16]. While such abstractions accelerate learning, they implicitly constrain the agent to the behaviors envisioned by the designer rather than those dictated by the underlying process physics. To move beyond these assumptions while retaining tractable training cost, optimization can couple learning directly to a physics-based transient thermal solve, even if the process model is still simplified.

In this paper, we present a physics-informed reinforcement learning framework for adaptive toolpath planning that couples learning with finite-element simulations of transient heat conduction. The framework is intentionally positioned as an intermediate step between purely geometric or heuristic RL environments and full three-dimensional process models. Even though the domain is two-dimensional and single-layer, each action is evaluated against an evolving thermal field. This integration enables the agent to learn physically consistent behaviors while remaining sensitive to local temperature variations. To promote robustness and assess generalization, we employ procedural generation to randomize part geometry and start position.

The paper begins by reviewing the powder bed fusion process and establishing a finite-element thermal model that captures heat transfer during layer-wise manufacturing. Building on this physical foundation, two complementary reinforcement learning environments are introduced: a geometric model that isolates path planning behavior, and a thermal model that builds on the geometric model by incorporating spatially resolved temperature feedback. Both environments use procedurally generated targets to ensure robust generalization. The reinforcement learning problem is then formulated with two representative algorithms, Deep Q-Network for value-based learning and Proximal Policy Optimization for policy gradient updates, benchmarked against a baseline scan strategy: the ZigZag path. Experimental results demonstrate that learned policies generalize effectively across diverse geometries while managing thermal accumulation. The discussion addresses computational requirements and outlines directions toward defect-aware objectives and industrial deployment.

2 Powder bed fusion modeling

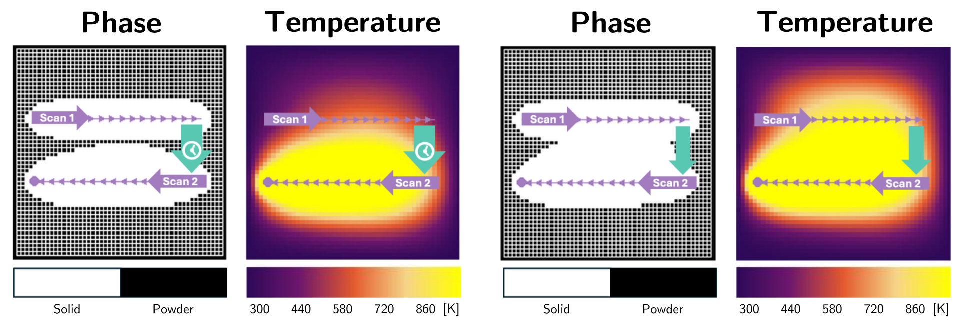
Powder bed fusion (PBF) is a layer-wise process in which a concentrated energy source selectively melts regions of a spread powder layer to form a solid, and successive layers build the part. Process setup typically starts by selecting the laser power and scan speed from a nominal process window. However, as geometries become more intricate, this simplified approach becomes insufficient. Hatch spacing, layer thickness, beam radius, absorptivity, heat conduction, and particularly scan ordering together govern local temperature peaks, gradients, and thus the print quality [1, 2, 4, 5].

2.1 Powder bed fusion process

Traditionally, in laser powder bed fusion, the laser can be modelled as a heat source with a Gaussian intensity distribution. A commonly used scalar process metric to determine the process settings is the line energy density,

$$E_{\text{line}} = \frac{\alpha_{\text{abs}} P}{v \ell_h}, \tag{1}$$

Where α_{abs} is the absorptivity, P is the incident power, v is the laser velocity, and ℓ_h is the lateral offset between adjacent tracks. This metric assumes thermal independence and ignores the time between passes.



(a) Longer waiting time; the material between lines has solidified.

(b) Shorter waiting time; the material between lines is partly molten.

Fig. 1 Two scan lines with identical line energy density but different waiting times between the first and second scan

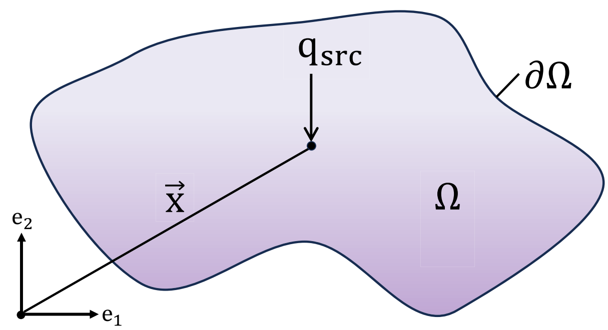


Fig. 2 Schematic representation of the thermal boundary value problem

However, this timing controls the heat-affected zone (HAZ) width. Short pauses limit cooling and raise the baseline temperature, broadening the HAZ. Figure 1 compares two scan lines with the same line energy density but different waiting times between the first and second scan. In Fig. 1a, the second line is scanned after a longer waiting time, so the first track has cooled and the material between the lines is mostly solid. In Figure 1b, the second line is printed after a much shorter waiting time. This means the material between the lines is partly molten, the melt pools overlap more, and a larger region between tracks is remelted. Thus, by simply changing the time between passes changes the phase state between neighboring lines and the final track shape, even though the scalar process metric E_{line} is identical in both cases. This motivates reinfrocement learning that responds to the local temperature field rather than using fixed raster patterns.

2.2 Finite element thermal model

To capture these thermal dynamics, we model heat transfer using a finite-element framework. The temperature field $T(\mathbf{x}, t)$ is computed on a planar domain $\Omega \subset \mathbb{R}^2$, where $\mathbf{x} = (x, y) \in \Omega$ denotes the position of the Gaussian heat source, as illustrated schematically in Fig. 2.

The temperature field is governed by the transient heat equation

$$\rho c \frac{\partial T(\mathbf{x},t)}{\partial t} - \nabla \cdot (\kappa \nabla T(\mathbf{x},t)) + h\,(T(\mathbf{x},t) - T_\infty) = q_{\text{src}}(\mathbf{x},t) \text{ in } \Omega, \qquad T(\mathbf{x},t) = \bar{T} \quad \text{on } \partial\Omega. \tag{2}$$

Here, ρc is the volumetric heat capacity with ρ the material density and c the specific heat, κ is the thermal conductivity of the material, and $q_{\text{src}}(\mathbf{x}, t)$ is the volumetric heat-source rate produced by the moving laser. The boundary condition enforces a fixed temperature $\bar{T}$ on the in-plane boundary $\partial\Omega$, while the term $h\big(T(\mathbf{x}, t) - T_\infty\big)$ models effective convective heat loss through the out-of-plane surfaces, with T_∞ the ambient temperature and h an effective out-of-plane heat-transfer coefficient. This model is intentionally simplified. It captures transient heat conduction and scan-order effects, while omitting latent heat, temperature-dependent material properties, Marangoni flow, vaporization, recoil effects, and full melt-pool fluid dynamics.

2.2.1 Heat source model

The heat source $q_{\text{src}}(\mathbf{x}, t)$ in point $\mathbf{x}$ and time t is modeled as a movable two-dimensional Gaussian distribution described by the magnitude of the rate of the laser beam:

$$q_{\text{src}}(\mathbf{x}, t) = \frac{P}{\pi\sigma^2} \exp\left(-\frac{|\mathbf{x} - \mathbf{x}_{\text{s}}(t)|^2}{2\sigma^2}\right), \tag{3}$$

Where P is the laser power, σ is the Gaussian beam width, and $\mathbf{x}_{\text{s}}(t) \in \Omega$ is the time-varying beam position controlled by the agent. To ensure energy conservation when the beam approaches domain boundaries, the fraction of power that lies within the computational domain is computed

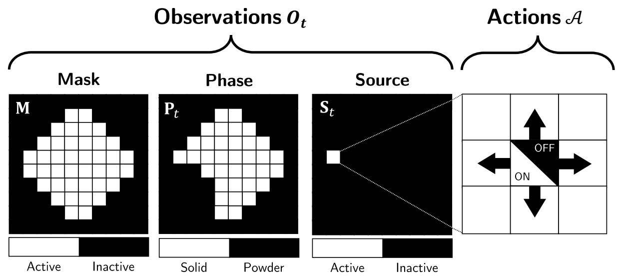


Fig. 3 Geometric environment with action and observation space. The legend distinguishes inactive background, the binary mask, the phase state (powder or melted), and the source state

analytically using the cumulative distribution function of the standard normal distribution. When discretizing the heat source onto the finite-element mesh, the nodal heat loads are then scaled such that the total integrated power remains P regardless of beam position, preventing artificial power variations that would otherwise introduce numerical artifacts. In the present study, P and σ are fixed to isolate toolpath sequencing and source on/off control; variable-power control is left for future work.

2.2.2 Discretization

To numerically solve the transient heat equation, we discretize the spatial domain Ω using a structured mesh of bilinear quadrilateral finite elements. The continuous temperature field $T(\mathbf{x}, t)$ is approximated by nodal values $\mathbf{T} \in \mathbb{R}^{n_{\text{dof}}}$, where n_{dof} is the total number of degrees of freedom. Within each element, temperature is interpolated via bilinear shape functions. For time integration, we employ the first-order implicit (backward) Euler method with time step Δt, updating $\mathbf{T}$ at each step. This finite element scheme accurately captures heat accumulation and cooling, enabling scan sequences that adapt to the evolving temperature field.

3 Environments

To explicitly capture the thermal effects described above, we couple a physics-based model with a reward signal that evaluates how each action reshapes the temperature field. Our approach proceeds in two stages using custom-designed deterministic square grid environments with N elements per side. First, the geometric environment serves as a lightweight sandbox that abstracts away thermal physics, focusing exclusively on time-optimal path planning for rapid iteration and hyperparameter tuning. Second, the thermal environment builds upon this foundation by incorporating the finite-element heat model, enabling realistic simulation of heat accumulation and cooling between passes to more faithfully mimic powder bed fusion processes. Although this thermal environment remains two-dimensional and single-layer, it provides physics-informed state transitions instead of heuristic thermal updates. In both environments, binary masks indicate valid processing regions, with targets generated procedurally at runtime to ensure robust adaptation across diverse input configurations.

Table 1 Reward function weights

Weight	Description	Value
α_f	Reward for covering target area	1.5
α_s	Penalty for depositing outside target	−0.15
α_r	Penalty for revisiting covered cells	−0.15
α_t	Time penalty per action	−1.5

3.1 Geometric environment

The geometric environment serves as the foundational model where we optimize the toolpath with respect to minimal time required to complete a layer, deliberately abstracting away thermal conduction to isolate the path planning problem. The environment defines a two-dimensional square grid with N^2 elements, where each element represents a printable region. The agent selects from a discrete action space $\mathcal{A}$, consisting of five available actions: four movement directions (up, down, left, right) and a source toggle (on or off) as illustrated in Fig. 3.

The environment observation is defined as:

$$\mathbf{O}_t = [\ \mathbf{M},\quad \mathbf{P}_t,\quad \mathbf{S}_t,\ ]^\top \in \mathbb{R}^{3\times N^2}, \tag{4}$$

Where $\mathbf{M} \in \{0,1\}^{N^2}$ is the target-mask channel, $\mathbf{S}_t \in \{0,1\}^{N^2}$ is a one-hot source channel indicating the laser location and state, and $\mathbf{P}_t \in \{0,1\}^{N^2}$ is the cumulative phase map at time t constructed by the incremental phase:

$$\Delta\mathbf{P}_t \in \{0,1\}^{N^2}, \tag{5}$$

Where $\Delta\mathbf{P}_t$ marks the newly phase changed cells. The cumulative phase channel is then updated as

$$\mathbf{P}_t = \max_{0\le\tau\le t} \Delta\mathbf{P}_\tau, \qquad \mathbf{P}_0 = \mathbf{0}. \tag{6}$$

Hence, $\mathbf{P}_t$ encodes the full printing history up to the current timestep, allowing to identify covered and uncovered regions during learning. The learning is driven by a composite reward

$$G = \sum_{t=0}^{H} \left(\alpha_\mathrm{f}\, r_t^{\mathrm{fill}} + \alpha_\mathrm{s}\, r_t^{\mathrm{spill}} + \alpha_\mathrm{r}\, r_t^{\mathrm{refill}} + \alpha_\mathrm{t}\, r_t^{\mathrm{step}} \right)$$

where G is the cumulative episode return, r_t^{fill} rewards depositing within the target mask, r_t^{spill} penalizes spill outside the target, r_t^{refill} penalizes redundant revisits, and r_t^{step} applies a per-step penalty that biases the policy toward fast coverage within minimal time. Each weight $\alpha_i \in \{\alpha_\mathrm{f}, \alpha_\mathrm{s}, \alpha_\mathrm{r}, \alpha_\mathrm{t}\}$ is treated as a hyperparameter tuned via hyperparameter optimization to balance these competing objectives by optimizing for a metric which is independent of the rewards using Tree-structured Parzen Estimator [18]. Table 1 shows the final tuned weights.

3.2 Thermal environment

Building upon the geometric foundation, the thermal environment reintroduces the thermal physics that were deliberately omitted in the first stage. This model extends the basic setup by incorporating thermal feedback to mimic powder bed fusion processes more realistically, while preserving the same action and observation layout with an added temperature channel as shown in Fig. 4 and defined as:

$$\mathbf{O}_t = [\ \mathbf{M},\quad \mathbf{S}_t,\quad \mathbf{T}_t,\quad \mathbf{P}_t\ ]^\top \in \mathbb{R}^{4\times N^2}. \tag{7}$$

Here, $\mathbf{T}_t$ is the current temperature field computed using the finite-element model described in Sect. 2, providing the agent with spatially resolved thermal information.

The interior raster is a square grid with N^2 quadrilateral elements (cells), where each element is defined by four corner nodes. Because adjacent elements share nodes, an $N \times N$ element mesh requires $(N+1)\times(N+1)$ nodes. To enforce boundary conditions, the assembled finite-element grid includes one padded layer of elements on all sides, increasing the total number of nodal unknowns to

$$n_{\mathrm{dof}} = (N+3)^2 .$$

These auxiliary boundary elements are shown only in demo visualizations and are omitted from the schematic observation-space figures, as they serve purely numerical purposes and are not used directly in toolpath optimization.

As in the geometric environment we maintain a binary coverage map, but here phase changes are recorded only when the local temperature exceeds a melt threshold T_m. In this simplified setting, T_m is an effective threshold for

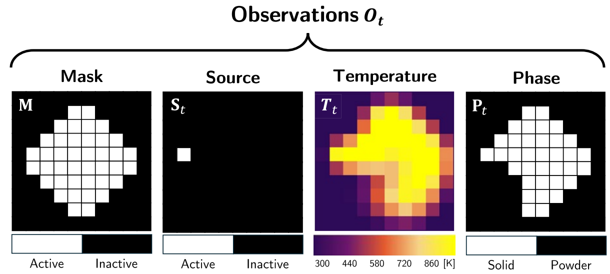


Fig. 4 Thermal environment observation space

Table 2 Thermal environment model parameters used in all simulations

	Description	Value
Domain size	$L_x \times L_y$	1.0 mm × 1.0 mm
Grid size	$N \times N$	10 × 10
Thermal conductivity	κ	113 W m^{-1} K^{-1}
Heat transfer coeff	h	0 W m^{-2} K^{-1}
Convection ambient temp. (inactive since $h = 0$)	T_∞	300 K
Vol. heat capacity	ρc	2.403 × 10^{6} J m^{-3} K^{-1}
Powder boundary reference temperature	$\bar{T}$	300 K
Initial temperature	T_0	300 K
Melt-threshold parameter	T_m	850 K
Source power	P	3.0 × 10^{6} W m^{-1}
Source speed	v	10 ms^{-1}
Gaussian width	σ	5.0 μm
Time step	Δt	1.0 × 10^{-5} s

updating the binary phase map rather than the literal melting temperature of a specific engineering alloy. Leading to the phase update:

$$\Delta\mathbf{P}_t = \mathbf{1}\{\mathbf{T}_t > T_\mathrm{m}\} \in \{0,1\}^{N^2} . \tag{8}$$

The material and simulation parameters used for the thermal environment are summarized in Table 2. These values define an effective surrogate parameter set for the simplified thermal benchmark and are not calibrated to a single engineering alloy.

In the reported setup, $h = 0$ disables the convective loss term, so T_∞ does not influence the solution. Heat dissipation is therefore governed primarily by conduction into the padded inactive region surrounding the printable domain, which is held at $\bar{T} = 300\,\mathrm{K}$.

3.3 Procedural randomization

To assure generalization, both environments draw their initial targets from a procedural generator [19]. Specifically, we sample shapes g, sizes l, rotations θ, and starting positions $\mathbf{p}$ at the beginning of each episode. The initial observation $\mathbf{O}_0$ therefore consists of a mask channel $\mathbf{M}(g, l, \theta)$, a source channel $\mathbf{S}_0(\mathbf{p})$, and a zeroed phase channel:

$$\mathbf{O}_0 = [\ \mathbf{M}(g, l, \theta) \quad \mathbf{S}(\mathbf{p}) \quad \mathbf{0}\ ]^\top . \tag{9}$$

The initial source position is sampled as

$$\mathbf{p} \sim \mathcal{P} \subset \mathbb{R}^2. \tag{10}$$

Here, $\mathcal{P}$ denotes the set of source positions including positions centered within the grid cells. The mask is parameterized by the discrete shape identifier $g \in \mathcal{G}$, a continuous size factor $l \in \mathcal{L}$, and a rotation angle $\theta \in \Theta$:

$$g \sim \mathcal{G}, \quad l \sim \mathcal{L} \subset [0, 1], \quad \theta \sim \Theta \subset [0, 360] .$$

where $\mathcal{G}$ is the set of admissible shapes, $\mathcal{L}$ is the set of size factors, and Θ is the set of angles. Together, the sets $\mathcal{P}, \mathcal{G}, \mathcal{L}, \Theta$ define the environment distribution from which training rollouts are drawn and held-out validation/test instances are sampled. Table 3 lists the sampled ranges.

Table 3 ProcGen parameter ranges for environment randomization

	Train	Validation	Test
$\mathcal{P}$ (position)	$[1.5 \times 10^{-4}, 1.05 \times 10^{-3}]^2$	$(1.5 \times 10^{-4}, 1.5 \times 10^{-4})$	$(1.5 \times 10^{-4}, 1.5 \times 10^{-4})$
$\mathcal{G}$ (shapes)	26 letters	10 symbols	6 shapes
$\mathcal{L}$ (size)	[0.1, 1]	[0.1, 1]	[0.1, 1]
Θ (angle)	[0, 360]	[0, 360]	0

$[a, b]^2 \equiv [a, b] \times [a, b]$. x and y are sampled independently

4 Reinforcement learning

Using the environments described above, we optimize toolpaths with reinforcement learning (RL). Each timestep t follows a loop: observe the current grid observation $\mathbf{O}_t$, choose a discrete action a_t, receive a scalar reward r_t, and advance to the next observation $\mathbf{O}_{t+1}$. The policy $\pi(a_t \mid \mathbf{O}_t)$ encodes the decision rule as shown in Fig. 5.

From a formal perspective, RL is typically defined on a Markov Decision Process (MDP) with latent states s_t and actions a_t, and extensions to partially observable MDPs when the agent only sees indirect measurements. In our implementation, the agent interacts solely with the observable grid tensors described in Sect. 3 (masks, source position, phase, and temperature), so all policies and value functions below are written in terms of the observation $\mathbf{O}_t$. When this observation fully captures the underlying physical configuration it coincides with the MDP state; if not, the setting is effectively partially observable.

The learning goal for the agent is to tune this policy to maximize the discounted sum of future rewards:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] . \tag{11}$$

The expectation operator $\mathbb{E}$ averages over all possible future trajectories under the current policy π, and the discount factor $\gamma \in [0, 1)$ tempers distant rewards so the optimization balances immediate progress with long-term efficiency.

In this study we focus on two complementary RL algorithms that suit the discrete-action structure: Deep Q-Network (DQN), which learns an action-value map to rank possible moves, and Proximal Policy Optimization (PPO), which updates a stochastic policy by taking cautious gradient steps. Both algorithms rely on tunable hyperparameters and the reward weights introduced earlier. We select these via hyperparameter optimization to ensure stable learning and good generalization across diverse targets.

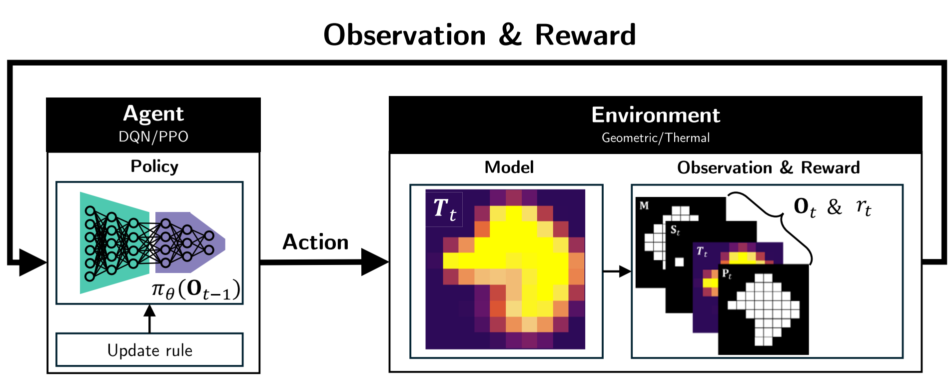


Fig. 5 Simplified overview of the reinforcement learning training loop, illustrating how policy updates and environment interactions mutually influence learning

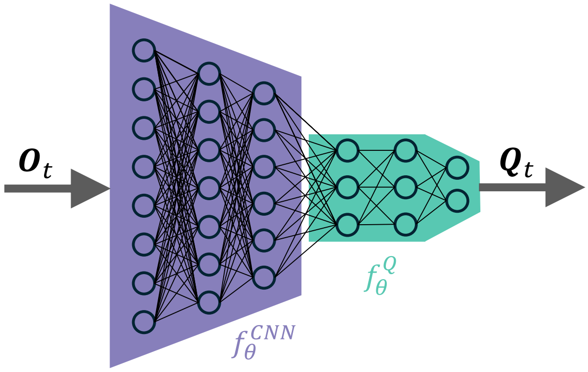


Fig. 6 DQN architecture

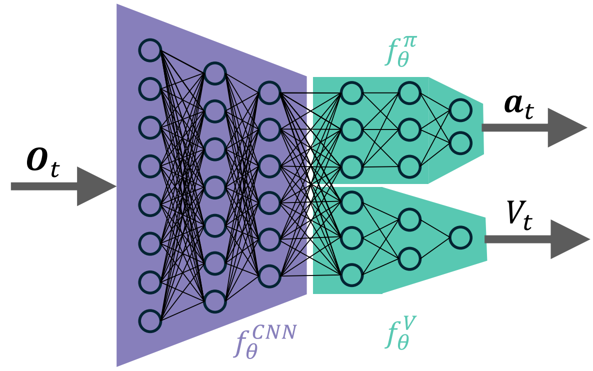


Fig. 7 PPO architecture

Table 4 Final DQN hyperparameters

Parameter	Value
Learning rate (η)	9.0×10^{-5}
Discount factor (γ)	0.995
Replay buffer size ($\mathcal{N}$)	2×10^5
Exploration parameter (ε)	$1.0 \rightarrow 0.05$
Target network update (τ_{target})	2000
Feature extractor (f_θ^{CNN})	[512, 512, 512]
Head architecture (f_θ^{Q})	[32]

Table 5 Final PPO hyperparameters

Parameter	Value
Learning rate (η)	4.7×10^{-4}
Discount factor (γ)	0.995
Clip range ($\varepsilon_{\text{clip}}$)	0.1
GAE lambda (λ)	0.8
Feature extractor (f_θ^{CNN})	[128, 256]
Head architecture (f_θ^{π}, f_θ^{V})	[128, 128]

4.1 Deep Q-Network

Deep Q-Network is a value-based, off-policy method that approximates the optimal action-value function $Q^*(\mathbf{O}_t, a_t)$, satisfying the Bellman equation [20]:

$$Q^*(\mathbf{O}_t, a_t) = \mathbb{E}_{\mathbf{O}_{t+1}}\left[r_t + \gamma \max_{a_{t+1}} Q^*(\mathbf{O}_{t+1}, a_{t+1})\right]. \tag{12}$$

The equation states that the value of taking an action equals the immediate payoff plus the best future payoff we can expect after that move, discounted to reflect that rewards further in time count slightly less. This Q-value function is approximated using a neural network parameterized by θ, taking as input the observation $\mathbf{O}_t$ and outputting a vector $Q(\mathbf{O}_t)$ of action-values. A schematic illustration is depicted in Fig. 6, where $f_\theta^{\text{DQN}}(\mathbf{O}_t) = f_\theta^{\text{Q}}(f_\theta^{\text{CNN}}(\mathbf{O}_t))$.

Actions are selected via an ε-greedy strategy to balance exploration and exploitation. At test time, the action with the highest value is then selected during operation:

$$a_t^* = \arg\max_{a_t} Q(\mathbf{O}_t, a_t).$$

To stabilize learning, DQN employs a target network and samples mini-batches of transitions from a replay buffer. The target network is a periodically (every τ_{target} steps) updated copy of the online network used to compute fixed Q-value targets. This reduces oscillations when updating the main network. Key algorithm-specific hyperparameters include the exploration parameter ε with its decay schedule, the replay-buffer capacity $\mathcal{N}$, and the target-network update frequency τ_{target}. The final hyperparameter settings are summarized in Table 4.

4.2 Proximal policy optimization

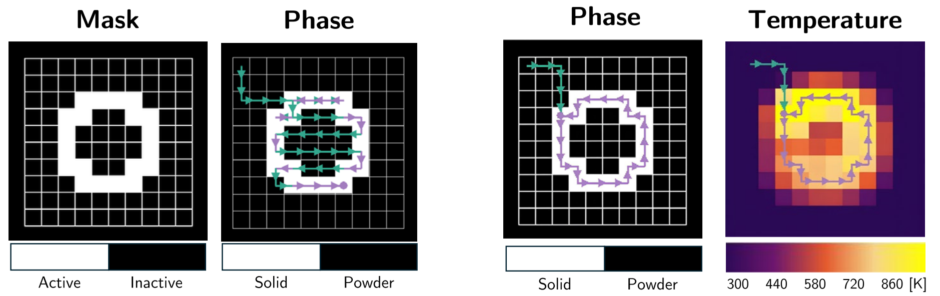
Whereas DQN reasons through a value function and replayed experience, Proximal Policy Optimization (PPO) takes the complementary route of updating a stochastic policy $\pi_\theta(a_t \mid \mathbf{O}_t)$ directly [21]. Instead of driving the network toward a bootstrapped value target, PPO adjusts the policy parameters themselves while constraining each step to remain close to the previous iterate for stability. This balance is enforced through clipped importance sampling ratios:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t A_t, \text{clip}\left(r_t, 1-\varepsilon_{\text{clip}}, 1+\varepsilon_{\text{clip}}\right) A_t\right)\right], \tag{13}$$

where $r_t = \frac{\pi_\theta(a_t|\mathbf{O}_t)}{\pi_{\theta_{\text{old}}}(a_t|\mathbf{O}_t)}$, and A_t is the estimated advantage.

This objective compares the likelihood of the sampled action under the updated versus previous policy, clipping the ratio so the update stays close to the old behavior. The

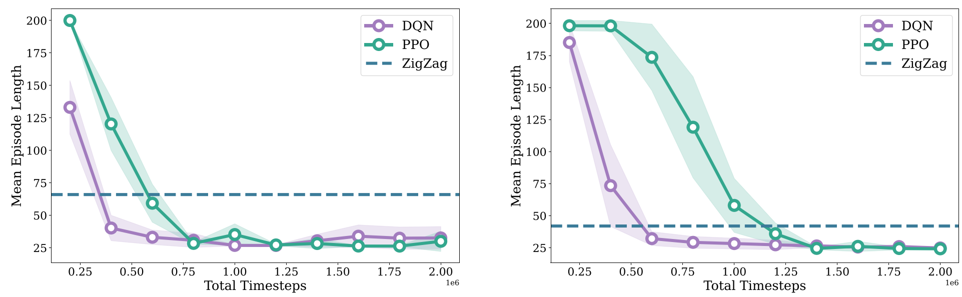
Fig. 8 Toolpath comparison on different environments. Black denotes inactive background, light cells denote printable regions, purple line segments denote source-on motion, and green line segments denote source-off motion



(a) Zigzag toolpath on the Geometric environment.

(b) PPO-generated toolpath on the Thermal environment.

Fig. 9 Learning curves showing mean episode length over training steps for the validation (shaded bands: 95% CIs). DQN converges faster than PPO on both environments, with the speed gap most visible in the thermal environment



(a) Geometric environment

(b) Thermal environment

advantage term A_t weighs the step by how much the action beats its baseline, delivering a restrained, trust-region style policy update.

In this implementation, the PPO policy is parameterized by a convolutional neural network (CNN) f_θ^{CNN}, which branches into two heads: f_θ^π outputs action probabilities via a softmax layer, and f_θ^V predicts a scalar value $V(\mathbf{O}_t)$ for the current observation (i.e., the usual state-value in the MDP formalism), as illustrated in Fig. 7. The hyperparameters are listed in Table 5.

4.3 Reference policy

To contextualize the learned policies, we compare them against a deterministic Zigzag raster strategy that mirrors standard hatch patterns used in laser powder bed fusion. The Zigzag baseline sweeps row by row across the grid, alternating scanning direction between successive rows. It follows a simple greedy rule: the energy source is toggled on only over unfilled target cells and remains off elsewhere, so it never intentionally revisits already deposited cells. While this makes Zigzag fast on simple geometries, it struggles on intricate shapes requiring continuous paths, where the row-wise structure forces extra repositioning and longer episodes. Figure 8 visualizes the Zigzag strategy alongside a PPO-generated toolpath.[1]

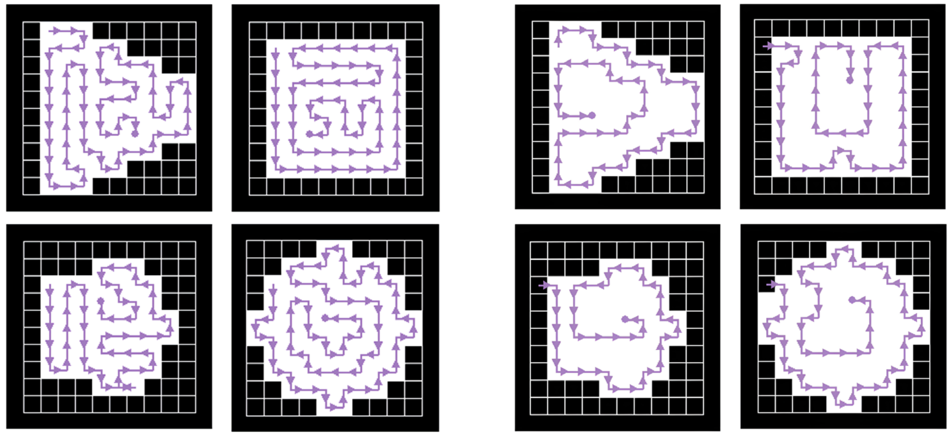
4.4 Metrics

For each environment, we train 10 independent seeds and evaluate each seed over 250 episodes. This sample size keeps the 95% confidence-interval relative half-width (RHW) below 15% for all reported metrics. For each seed, we take the median episode reward and median episode length (steps to completion) to reduce the impact of occasional runs that hit the step limit of 200 steps. We then report the mean of these per-seed medians and the corresponding 95% confidence-interval width (CIW), estimated by non-parametric bootstrapping over episodes. In addition to episode length, we report a speed metric defined as the fraction of target cells filled per step, making it comparable across targets of different sizes and topology.

5 Results

With the training setup defined, we now examine how the learned policies behave. We report metrics on both the geometric and thermal environments to investigate how the value-based DQN and the policy-gradient PPO translate their different update rules into toolpath quality.

[1] https://github.com/rschmeitz/ctrlp-zoo



(a) Generated paths in the Geometric environment.

(b) Generated paths in the Thermal environment.

Fig. 10 Learned toolpaths for identical masks in the geometric versus thermal environment, showing how an extended, diffusive heat source in the thermal model leads the policy to scan boundary regions first to preheat the interior before a final interior pass, whereas in the geometric model it mainly minimizes travel distance with compact interior routes

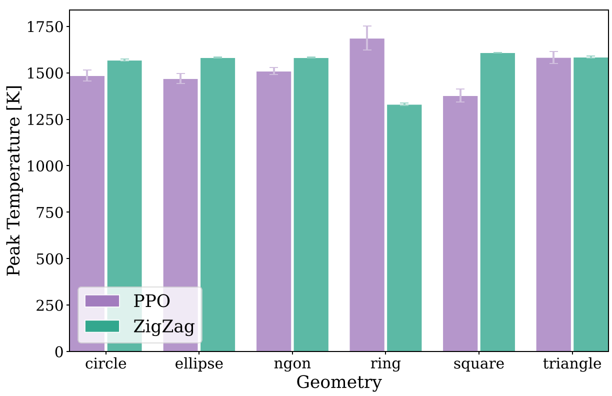


Fig. 11 Peak temperatures in the thermal environment on the test masks. Zigzag generally overheats due to repeated rescanning, except for the ring geometry where frequent laser off periods limit thermal accumulation

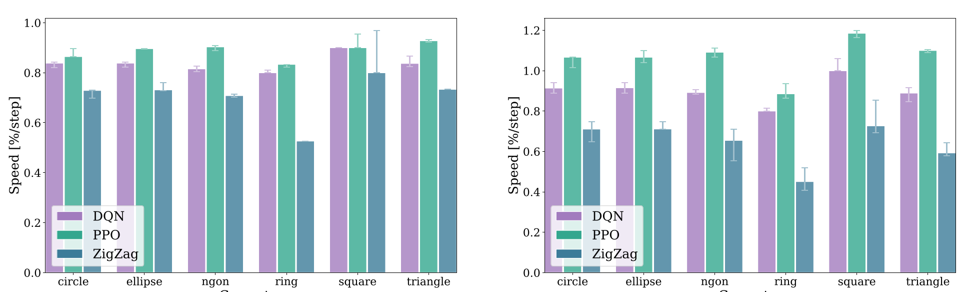
5.1 Policy performance

Learning dynamics differ significantly between DQN and PPO. Across both environments, DQN converges in fewer steps than PPO, with the gap most pronounced in the thermal environment, reflecting its higher sample efficiency enabled by experience replay. This can be seen in the learning curves in Fig. 9, which summarize the mean episode length, with shaded regions denoting 95% confidence intervals.

The effect of the thermal diffusion on path planning is illustrated in Fig. 10, which compares learned paths on identical masks for the geometric versus the thermal environment. In the geometric environment, the source footprint spans a single cell and the policy primarily minimizes travel distance, producing compact interior routes. In the thermal environment, the Gaussian heat source covers multiple cells and heat diffuses through the substrate, so phase changes along the boundary preheat interior regions. As a result, the thermal policy preferentially scans exterior regions before proceeding inward, and the final interior pass can strike through the middle rather than visiting every cell individually while still achieving full melt.

5.2 Thermal performance

Physical behavior in the thermal environment is evaluated using the peak temperature reached during toolpath execution, as shown in Fig. 11. We use peak temperature as the primary physical indicator because it directly reflects overheating risk while remaining comparable across policies and geometries in this benchmark setting. For most geometries, the Zigzag strategy produces the highest peak



(a) Geometric environment

(b) Thermal environment

Fig. 12 Generalization performance on a 10×10 grid for the test shapes. Both RL agents (DQN and PPO) consistently achieve higher task completion speeds (cells filled per step) than the Zigzag baseline across diverse test geometries, demonstrating robust generalization and adaptability

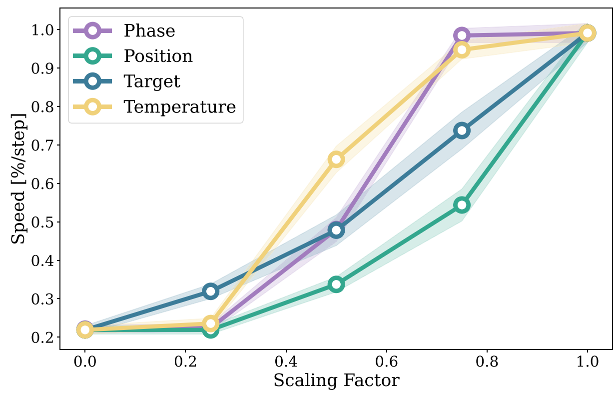


Fig. 13 Task completion speed versus observation channel scaling factor on the test masks, evaluated using the PPO policy. Reduced observation information leads to slower task execution

temperatures, reflecting repeated rescanning of fixed lines without accounting for the evolving thermal field. An exception is the ring geometry, where Zigzag exhibits lower peak temperatures due to frequent laser off periods and the absence of a continuous scan path.

Both learning-based policies reduce peak temperatures by adapting the scan sequence to the local thermal state. PPO achieves the lowest peaks overall. In all cases, reduced peak temperatures are achieved without increasing path length, indicating that the improvements arise from adaptive ordering rather than trivial slowdowns. From a mechanical perspective, the lower peak temperatures imply a reduced risk of overheating-induced defects.

5.3 Generalization

We assess how well trained policies transfer from training shapes to held-out test masks in both environments by comparing the speed metric across the held-out shape sets.

Figure 12 summarizes the generalization performance on the test shapes. Both DQN and PPO achieve higher task completion speeds (measured as filled cells per step) compared to the Zigzag baseline, which performs particularly poorly on irregular geometries such as rings. PPO demonstrates a slight advantage over DQN, likely due to its ability to optimize longer action sequences more effectively through direct policy updates.

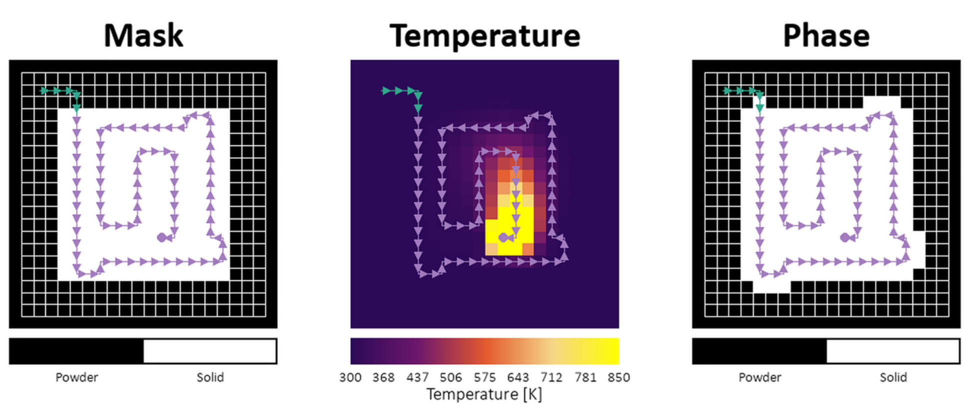
5.4 Observation sensitivity

Figure 13 reports task completion speed as a function of observation channel scaling, evaluated on the test masks using the trained PPO policy. Progressively reducing available state information slows task completion but maintains feasibility, indicating that richer observations improve efficiency rather than capability. Thermal information is clearly important for successful task execution, as evidenced by the consistent slowdown when it is reduced. However, it shows the least sensitivity to scaling compared to other channels, with the policy performing reasonably even at 0.5 amplification. This suggests the policy relies on relative temperature distributions to inform its decisions rather than absolute magnitudes under the present modeling assumptions. This should not be interpreted as evidence that temperature is unimportant in physical LPBF systems, where higher-fidelity physics and sensing would be required.

5.5 Scaling to higher resolution

To examine how the approach scales with resolution, we trained and evaluated policies on 20×20 grids. Figure 14 shows a representative PPO trajectory on a square geometry, confirming that learned policies complete higher-resolution targets effectively. However, because the agent cannot control the power or spread of the heat source, some cells outside the target mask are inevitably solidified, particularly at corners where the fixed Gaussian heat distribution extends beyond the desired boundary. Incorporating power modulation as an additional control dimension could improve geometric precision but would increase action space complexity and computational cost.

Fig. 14 PPO policy trajectory on a square geometry at higher resolution. The agent successfully completes the target but solidifies some boundary cells due to the fixed heat source amplitude



Additionally, training from scratch at a 20×20 mesh requires significantly more wall-clock time due to longer episodes and increased per-step simulator cost, reducing throughput from ~200 FPS to 90–120 FPS. Additional qualitative examples are provided in Appendix A.(Figs. 15, 16, 17)

6 Discussion

The results demonstrate that reinforcement learning can produce adaptive toolpath policies that outperform conventional zigzag strategies in both geometric and thermal environments. However, several considerations emerge regarding reward design, computational requirements, and practical deployment.

The current reward function prioritizes phase completion and speed but omits explicit defect penalties such as balling, warping, residual stress, temperature peaks, gradients, or cooling rates. While the learned policies implicitly manage thermal accumulation by adapting scan sequences, incorporating multi-objective rewards that explicitly penalize temperature peaks, gradients, or cooling rates may yield more physically robust strategies aligned with industrial quality requirements. Such extensions would enable the framework to address not only efficiency but also the microstructural and mechanical properties critical for real-world part quality.

Despite lightweight policies ($< 3 \times 10^6$ parameters), training is hardware intensive, requiring approximately 2 h per seed on an 18-core Intel Xeon Platinum 8360Y paired with an NVIDIA A100 GPU. Computational throughput, limited primarily by CPU-parallelized finite-element simulators, remains the dominant barrier to broader deployment. The observed scaling challenges at 20×20 resolution underscore this limitation, as longer episodes and increased per-step costs reduce throughput substantially. Practical directions to address this include surrogate modeling to approximate thermal responses more efficiently, multi-fidelity hierarchies that balance accuracy and speed, and warm-starting from lower-resolution policies to leverage transfer learning across grid scales. As ongoing future work, we are implementing a more sophisticated thermal model together with a faster solver and assembly pipeline to support larger-scale training experiments.

Translating these learned policies to industrial systems requires handling 3D geometries, varying boundary conditions, heterogeneous materials, and real-time sensor feedback. The present experiments are limited to single-layer 10×10 and 20×20 grids used as controlled benchmark settings. While procedural generation provides robustness to geometric variation within the training distribution, extending the framework to full-scale manufacturing environments remains essential for real-world adoption. Bridging the gap between the simplified 2D grid environment and practical deployment will necessitate not only computational advances but also integration with process monitoring and adaptive control systems that can respond to in-situ measurements.

7 Conclusion

We presented a reinforcement learning framework that couples procedurally generated toolpath tasks with a finite-element model to capture the physics governing powder bed fusion. By exposing agents to observation-based temperature feedback and randomized target geometries, the framework produces policies that adapt phase order to local heat accumulation rather than following pre-set rasters.

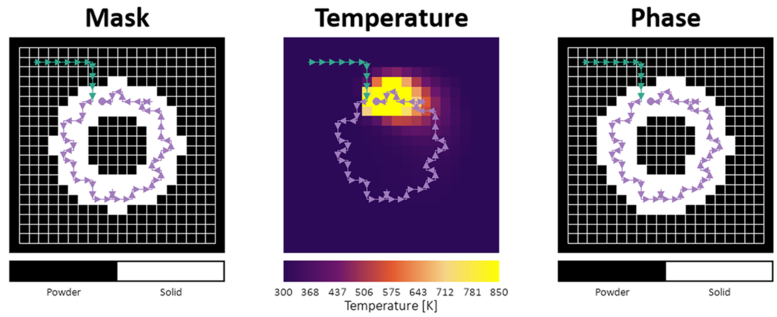
Across both environments, both DQN and PPO surpass the Zigzag baseline in coverage efficiency and robustness to irregular shapes. DQN benefits from rapid learning, whereas PPO retains higher generalization as the horizon and thermal complexity increase. These results indicate that on-policy methods better sustain performance when long-range thermal dependencies dominate, while off-policy value learning remains a strong option for sample efficient learning.

The study also underscores practical considerations: even with a small number of network parameters, attaining the reported gains required substantial CPU-driven simulator throughput complemented by GPU inference. Consequently, future extensions should target surrogate physics models or hierarchical curricula to reduce computational cost while integrating richer defect-aware rewards and ultimately scaling the approach to 3D toolpaths and variable boundary conditions. Experimental validation against printed parts and thermal measurements remains a key next step before practical deployment claims can be made.

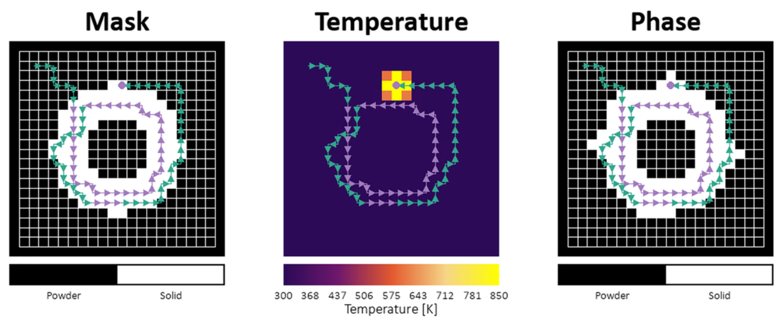
Appendix A. Thermal environment policy visualizations

See Figures

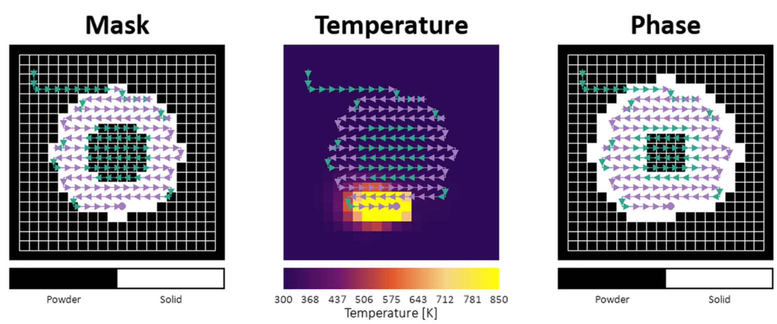
Fig. 15 Policy trajectories on ring geometry in the thermal environment



(a) PPO



(b) DQN



(c) ZigZag

Fig. 16 Policy trajectories on polygon geometry in the thermal environment

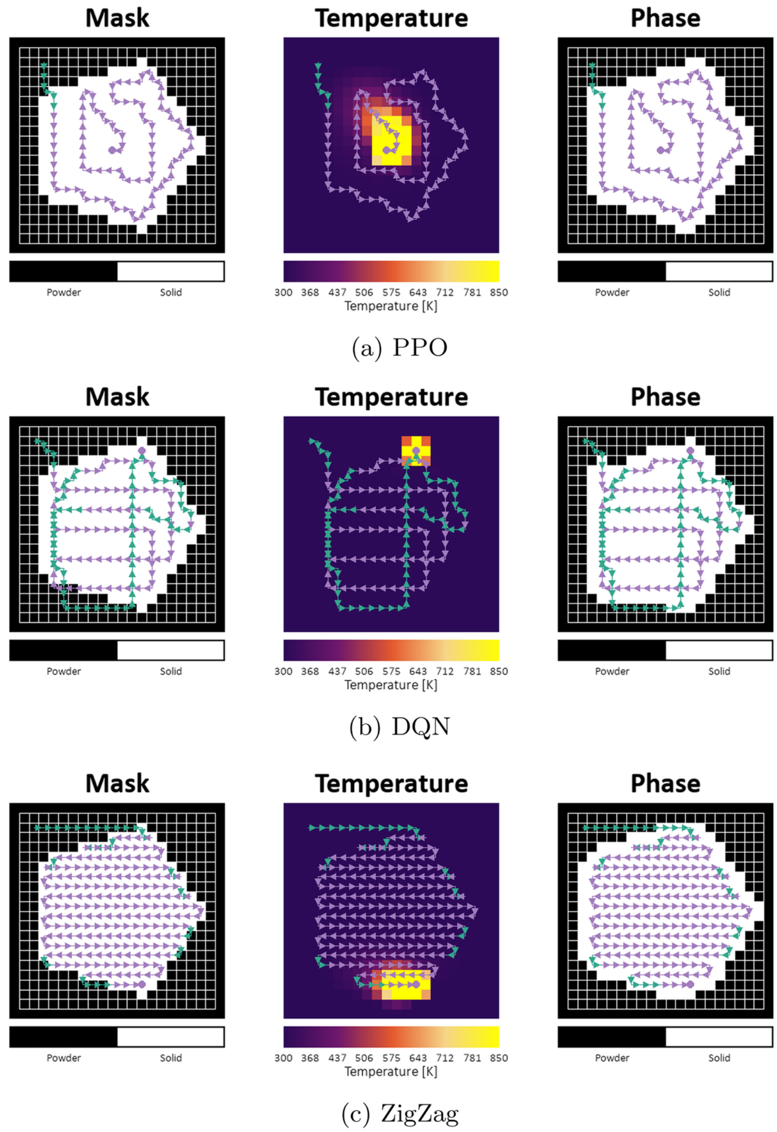
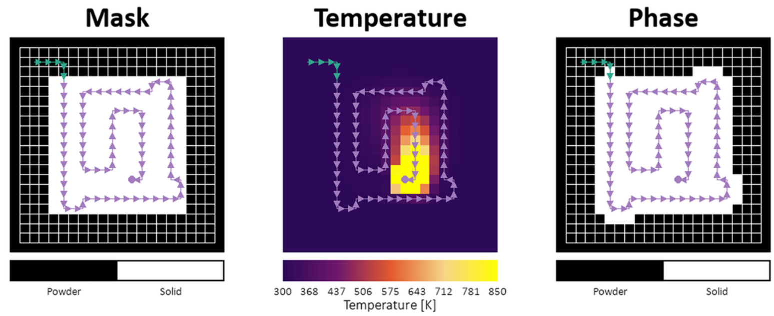
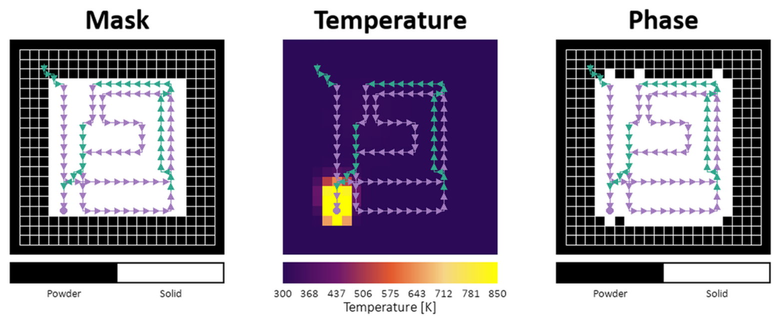


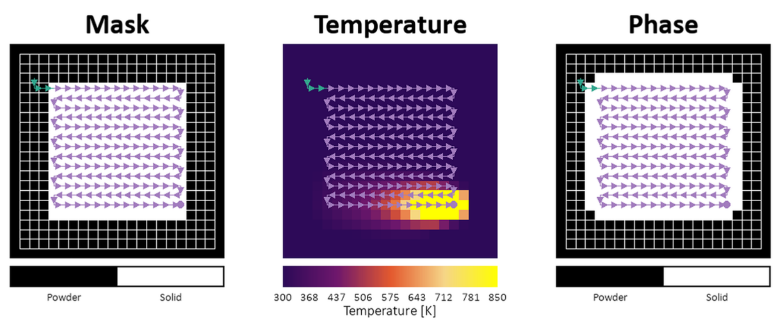
Fig. 17 Policy trajectories on square geometry in the thermal environment



(a) PPO



(b) DQN



(c) ZigZag

Acknowledgements This work was funded by the Eindhoven AI Systems Institute (EAISI). The initial draft of this manuscript was composed by the authors, which was subsequently refined and enhanced with the aid of OpenAI's ChatGPT [22].

Author contributions R.S. conceived the study, developed the methodology, implemented the software, performed the formal analysis and investigation, and wrote the original draft of the manuscript. R.J.M.W. contributed to the conceptualization and methodology, provided resources, supervised the research, acquired funding, and reviewed and edited the manuscript. J.J.C.R. contributed to the conceptualization and methodology, provided resources, supervised and administered the project, acquired funding, and reviewed and edited the manuscript. All authors reviewed and approved the final manuscript.

Data availability The data that support the findings of this study were generated through numerical simulations and reinforcement learning experiments. The datasets and code are available from the corresponding author upon reasonable request.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

1. Chen S-G, Gao H-J, Zhang Y-D, Wu Q, Gao Z-H, Zhou X (2022) Review on residual stresses in metal additive manufacturing: formation mechanisms, parameter dependencies, prediction and control approaches. J Market Res 17:2950–2974. https://doi.org/10.1016/j.jmrt.2022.02.054
2. Mostafaei A, Zhao C, He Y, Ghiaasiaan SR, Shi B, Shao S, Shamsaei N, Wu Z, Khouraytem N, Sun T, Pauza J, Gordon JV, Webler B, Parab ND, Asherloo M, Guo Q, Chen L, Rollett AD (2022) Defects and anomalies in powder bed fusion metal additive manufacturing. Curr Opin Solid State Mater Sci 26(2):100974. https://doi.org/10.1016/j.cossms.2021.100974
3. Ahmed N, Barsoum I, Haidemenopoulos GZ, Abu Al-Rub RK (2022) Process parameter selection and optimization of laser powder bed fusion for 316l stainless steel: a review. J Manuf Process 75:415–434. https://doi.org/10.1016/j.jmapro.2021.12.064
4. Drstvenšek I, Zupanič F, Bončina T, Pal S (2021) Influence of local heat flow variations on geometrical deflections, microstructure, and tensile properties of ti-6al-4v products in powder bed fusion systems. J Manuf Process 65:382–396. https://doi.org/10.1016/j.jmapro.2021.03.054
5. Munk J, Breitbarth E, Siemer T, Pirch N, Häfner C (2022) Geometry effect on microstructure and mechanical properties in laser powder bed fusion of ti-6al-4v. Metals 12(3):482. https://doi.org/10.3390/met12030482
6. Wang D, Yang Y, Yi Z, Zhang M, Hao L, Bai Y, Yang Y (2018) The effect of a scanning strategy on the residual stress of 316l steel parts fabricated by selective laser melting (slm). Materials 11(10):1821. https://doi.org/10.3390/ma11101821
7. Alsoufi MS, Elsayed AE (2018) Surface roughness quality and dimensional accuracy–a comprehensive analysis of 100% infill printed parts fabricated by a personal/desktop cost-effective FDM 3D printer. Mater Sci Appl 09(01):11–40. https://doi.org/10.4236/msa.2018.91002
8. Khrenov M, Templeton WF, Narra SP (2025) ADDOPT: An additive manufacturing optimal control framework demonstrated in minimizing layer-level thermal variance in electron beam powder bed fusion. J Manuf Sci Eng. https://doi.org/10.1115/1.4067325
9. Kim DH, Zohdi TI (2021) Tool path optimization of selective laser sintering processes using deep learning. Comput Mech 67(4):772–789. https://doi.org/10.1007/s00466-021-02079-1
10. Jiang J, Ma Y (2020) Path planning strategies to optimize accuracy, quality, build time and material use in additive manufacturing: a review. Micromachines 11(7):633. https://doi.org/10.3390/mi11070633
11. Hong G, Yang Z, Lu Y, Lane B, Yeung H, Kim J (2024) Multiscale model predictive control for laser powder bed fusion additive manufacturing. In: Proceedings of the ASME 2024 international design engineering technical conferences and computers and information in engineering conference (IDETC/CIE 2024). ASME, Washington, DC, USA. https://doi.org/10.1115/DETC2024-143601
12. Ogoke F, Farimani AB (2021) Thermal control of laser powder bed fusion using deep reinforcement learning. Addit Manuf 46:102033. https://doi.org/10.1016/j.addma.2021.102033
13. Qin M, Ding J, Qu S, Song X, Wang CCL, Liao WH (2024) Deep reinforcement learning based toolpath generation for thermal uniformity in laser powder bed fusion process. Addit Manuf 79:103937. https://doi.org/10.1016/j.addma.2023.103937
14. Dharmadhikari S, Menon N, Basak A (2023) A reinforcement learning approach for process parameter optimization in additive manufacturing. Addit Manuf 71:103556. https://doi.org/10.1016/j.addma.2023.103556
15. Mozaffar M, Ebrahimi A, Cao J (2020) Toolpath design for additive manufacturing using deep reinforcement learning. https://doi.org/10.48550/arXiv.2009.14365. arXiv:2009.14365
16. Li X, Pattinson SW (2025) An efficient and uncertainty-aware reinforcement learning framework for quality assurance in extrusion additive manufacturing. Addit Manuf 110:104912. https://doi.org/10.1016/j.addma.2025.104912
17. Chung J, Shen B, Law ACC, Kong ZJ (2022) Reinforcement learning-based defect mitigation for quality assurance of additive manufacturing. J Manuf Syst 65:822–835. https://doi.org/10.1016/j.jmsy.2022.11.008
18. Watanabe S (2023) Tree-structured Parzen estimator: understanding its algorithm components and their roles for better empirical performance. https://doi.org/10.48550/arXiv.2304.11127
19. Cobbe K, Hesse C, Hilton J, Schulman J (2020) Leveraging procedural generation to benchmark reinforcement learning. In: Proceedings of the 37th international conference on machine learning (ICML). Proceedings of machine learning research, vol. 119, pp. 2048–2056. https://doi.org/10.48550/arXiv.1912.01588. arXiv:1912.01588
20. Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, Graves A, Riedmiller M, Fidjeland AK, Ostrovski G, Petersen S, Beattie C, Sadik A, Antonoglou I, King H, Kumaran D, Wierstra D, Legg S, Hassabis D (2015) Human-level control through deep reinforcement learning. Nature 518(7540):529–533. https://doi.org/10.1038/nature14236
21. Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O (2017) Proximal policy optimization algorithms. https://doi.org/10.48550/arXiv.1707.06347
22. OpenAI (2025) ChatGPT. https://chatgpt.com/. Accessed 14 Jan 2026